University of Pisa

Parallel and Distributed Systems

Optimization of Partitioned Hash Join with Duplicates

Assignment 1 - Report

Gianluca Panzani

April 8, 2026

Contents

1 Implementation **1**
1.1 Execution Variants 1
1.2 Grid Search approach 1
1.3 Hash Functions 2
1.3.1 Mask hashing 2
1.3.2 Xorshift hashing 2
1.3.3 Fold32 + fmix32 hashing 2

2 SIMD Optimization **2**
2.1 Compiler evidence of vectorization 2
2.2 AVX2 mask hash implementation 2

3 Results Analysis **3**
3.1 Throughput 3
3.2 Partition-time 4
3.3 Speedup 4

4 Conclusions **5**

1 Implementation

The implemented project focuses on the partition phase of a Partitioned Hash Join under duplicate-heavy workloads.

1.1 Execution Variants

The codebase provides a reproducible benchmarking pipeline to compare three execution variants: `plain_novec`, `plain_vec`, and `avx2`. Each variant is got by a different compilation of the same file (`main.cpp`) and applies the same high-level algorithm but uses different optimization strategies.

The executable targets can be built with the commands: `make plain_novec`, `make plain_vec` and `make avx2`.

Executing the programs as defined in the `README.md` file, they receive the same command-line arguments, which include:

- the dataset size `N`,
- the number of partitions `P`,
- the name of the hash function to be used,
- the output CSV file path.

The runtime workflow includes: loading the two input datasets, partitioning both relations with the selected hash function, computing throughput and checksums, writing partitioned binary outputs and appending one result row to the CSV file. This row contains the full experiment metadata (`N`, `P`, hash, execution type), timing metrics (partition time, global time) and other fields (like throughput and checksum), which are then used by the analysis notebooks to extract statistics.

1.2 Grid Search approach

The experimental automation is implemented with Bash orchestrators and Slurm runners. The script `run_grid_search.sh` performs:

- cleanup (`make cleanall`) and compilation (`make`),
- dataset generation job submission,
- chained Slurm submissions with dependencies for `plain_novec` $\rightarrow$ `plain_vec` $\rightarrow$ `avx2`.

The search space is defined in `runners/grid_config.sh`: $\texttt{N} \in \{10^5, 10^6, 10^7\}$, $\texttt{P} \in \{16, \ldots, 4096\}$ (powers of two), hash $\in \{\texttt{mask}, \texttt{xorshift}, \texttt{fmix32fold}\}$. This gives 81 configurations per executable for each full sweep.

The script `stability_benchmark.sh` uses `runners/grid_config_2.sh` (`N` fixed to 10^7) and repeats the same parameter grid `M` times (in the current dataset, 10 repetitions). This is used to separate raw performance from run-to-run variance.

An important implementation detail is that `compute_partitions_avx2` currently supports only `mask`. For unsupported hashes (`xorshift`, `fmix32fold`), the AVX2 executable exits early after the runtime check. Therefore:

- `results/results.csv` contains 81 rows for `plain_novec`, 81 rows for `plain_vec`, and 27 rows for `avx2`;
- `results/benchmark_results.csv` is used for stable repeated-run analysis.

This design makes experiments reproducible and auditable: build flags, execution logs, and vectorization diagnostics are persisted and tied to each benchmark batch.

1.3 Hash Functions

1.3.1 Mask hashing

This is the simplest mapping. It does not really mix the bits of the key: it just keeps the lowest bits by applying a bitwise AND with P - 1. So, logically, it is just a bit selection operation: the partition id is obtained from the least significant bits of the input key.

1.3.2 Xorshift hashing

This function performs a sequence of right shifts and XORs on the 64-bit key. The idea is to spread information from higher bits into lower bits, so that the final low bits used for partitioning depend on more than just the original least significant bits.

1.3.3 Fold32 + fmix32 hashing

This function first folds the 64-bit key into 32 bits by XORing the lower 32 bits with the upper 32 bits. Then it applies the fmix32 finalizer, which combines XORs, shifts and 32-bit multiplications to further scramble the bits. So, it first compresses the 64-bit value into 32 bits, then performs a stronger bit diffusion so that small differences in the input affect many output bits.

2 SIMD Optimization

2.1 Compiler evidence of vectorization

Vectorization is checked through GCC diagnostics generated at build time. In the Makefile, `plain_vec` and `avx2` are compiled with: `-fopt-info-vec-optimized -fopt-info-vec-missed`, while `plain_novec` is compiled with: `-fno-tree-vectorize`.

The following lines in `compilation/gcc_vec_report.txt` prove auto-vectorization for the `plain_vec` build for each hash function:

```
lib/partition.cpp:9:31: optimized: loop vectorized using 32 byte vectors
lib/partition.cpp:9:31: optimized: loop vectorized using 16 byte vectors
lib/partition.cpp:20:31: optimized: loop vectorized using 32 byte vectors
lib/partition.cpp:20:31: optimized: loop vectorized using 16 byte vectors
lib/partition.cpp:36:31: optimized: loop vectorized using 32 byte vectors
lib/partition.cpp:36:31: optimized: loop vectorized using 16 byte vectors
```

and the manual vectorization in the AVX2 path:

```
lib/partition_avx2.cpp:36:14: optimized: loop vectorized using 16 byte vectors
lib/partition_avx2.cpp:21:27: missed: couldn't vectorize loop
```

The missed one is the loop of the manual AVX2 implementation. This is expected, because the compiler does not recognize the custom intrinsics as a vectorizable pattern.

2.2 AVX2 mask hash implementation

The AVX2 partition kernel is implemented in `lib/partition_avx2.cpp`. The logic is the following:

1. Build SIMD constants: `mask_vec = broadcast(P-1)` and `perm_idx = [0,2,4,6,...]`.
2. Process four 64-bit keys per iteration (`__m256i` register):

- load 256 bits with `_mm256_loadu_si256`,
- compute partition ids via bitwise AND with `_mm256_and_si256`,
- pack low 32-bit lanes with `_mm256_permutevar8x32_epi32`,
- store 4 contiguous `uint32_t` outputs with `_mm_storeu_si128`.

3. Handle the tail (< 4 keys) with scalar code to preserve correctness for any input length.

3 Results Analysis

All quantitative values in this section are computed from `results/benchmark_results.csv`, because it is based on repeated runs and is more reliable than a single sweep.

3.1 Throughput

Figure 1 compares throughput over `P` for the three hash functions.

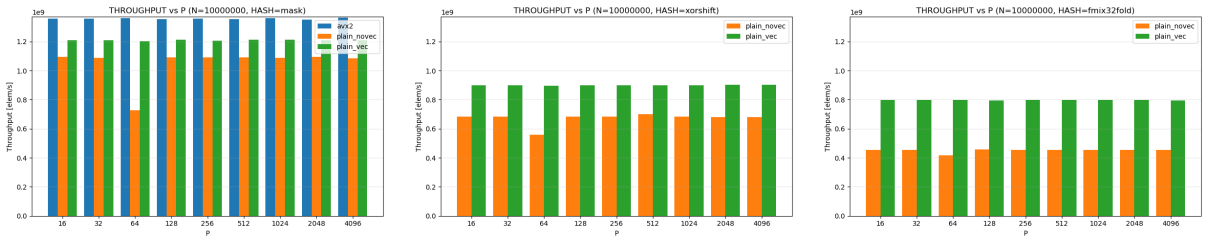

Figure 1: Throughput vs `P` for hash functions `mask`, `xorshift`, and `fmix32fold`.

The measured ordering is stable: `mask` is fastest, `xorshift` is second, `fmix32fold` is slowest. On average over `benchmark_results.csv`, throughput is:

- `plain_novec`: 1.08×10^9 elems/s (`mask`), 6.81×10^8 elems/s (`xorshift`), 4.54×10^8 elems/s (`fmix32fold`),
- `plain_vec`: 1.20×10^9 elems/s (`mask`), 8.98×10^8 elems/s (`xorshift`), 7.97×10^8 elems/s (`fmix32fold`),
- `avx2`: 1.36×10^9 elems/s (`mask` only).

This reflects the arithmetic complexity of the hash: `mask` is a simple bit operation, `xorshift` adds only shift/XOR steps and `fmix32fold` adds a stronger finalizer that includes 32-bit multiplications.

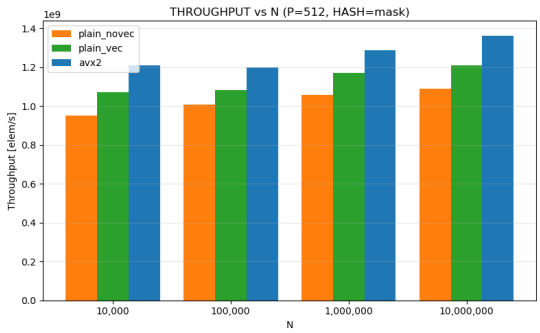

Figure 2: Throughput vs `N` for `mask`, with grouped bars for `plain_novec`, `plain_vec` and `avx2` at each `N`.

The throughput remains relatively stable across `N`, with a moderate increase for larger inputs. This suggests that fixed overheads are amortized as `N` grows, while asymptotic behavior stays close to linear.

3.2 Partition-time

Figure 3 shows partition time for each hash.

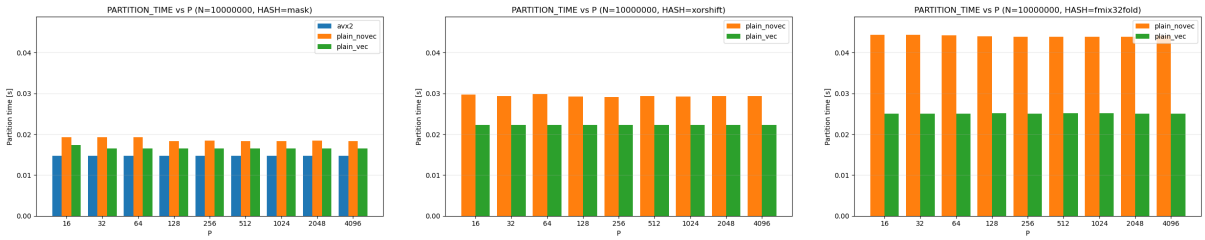

Figure 3: Partition time vs `P` for hash functions `mask`, `xorshift`, and `fmix32fold`.

Average partition time from `benchmark_results.csv` is coherent with throughput. The impact of `P` (with fixed `N`) is not so visible, rather than the impact of hash complexity.

3.3 Speedup

Figure 4 reports speedup over the scalar baseline (`plain_novec`).

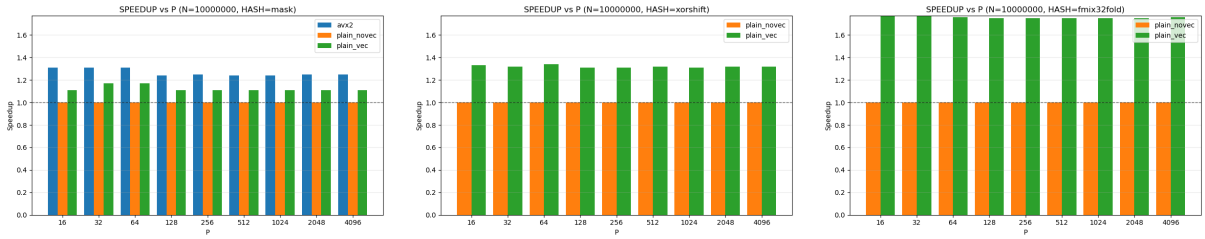

Figure 4: Speedup vs `P` (baseline: `plain_novec`) for hash functions `mask`, `xorshift`, and `fmix32fold`.

For `xorshift` and `fmix32fold`, AVX2 is currently unsupported; speedup is therefore driven by `plain_vec` only. In the repeated benchmark, `plain_vec` reaches about 1.32× on `xorshift` and 1.76× on `fmix32fold` versus `plain_novec`. Table 1 summarizes the `mask`-only repeated benchmark. Speedup is computed as:

$$\text{speedup}(\text{exec_type}) = \frac{\overline{T}_{\texttt{plain_novec}}}{\overline{T}_{\texttt{exec_type}}}$$

`exec_type`	`partition_time` [s]	`throughput` [elems/s]	`speedup`
`plain_novec`	0.018672333	1076883683.042	1.0000
`plain_vec`	0.016631667	1204691892.527	1.1227
`avx2`	0.014739778	1356894172.186	1.2668

Table 1: Average performance for `mask` over repeated runs (`benchmark_results.csv`).

4 Conclusions

The experiments confirm that partition performance is shaped by both algorithmic choices (`hash`, `P`) and execution strategy (`plain_novec`, `plain_vec`, `avx2`), with different factors dominating in different regions.

Hash selection has the strongest effect on absolute cost: `mask` is consistently the fastest, `xorshift` is intermediate, and `fmix32fold` is the most expensive due to stronger mixing.

SIMD still helps, but gains are not uniform. In the repeated benchmark, `plain_vec` improves over `plain_novec` by about $1.12\times$ on `mask`, $1.32\times$ on `xorshift`, and $1.76\times$ on `fmix32fold`. The manual AVX2 kernel gives the best `mask` throughput ($\approx 1.36 \times 10^9$ elems/s), with about $1.27\times$ speedup over scalar.

The number of partitions `P` still affects runtime, but in these results its impact is generally smaller than the choice of hash and execution kernel. At the same time, AVX2 support is currently limited to `mask`, so the fastest path is not yet available for the other hash functions.